AI-Powered IoT Plant Health Monitoring and Disease Detection

Sang Suh
Alex Bristow
East Texas A&M University
U. S. A.

Abstract

This paper presents a plant health monitoring system that combines environmental sensors and machine learning to detect plant stress and diseases. The system uses sensors such as the LM393 soil moisture sensor, Teyleten Robot pH sensor, DHT22/AM2302 humidity and temperature sensor, and the MSP3008 analog-to-digital converter, all connected to a Raspberry Pi. These sensors collect real-time data on soil moisture, pH, temperature, and humidity, essential for assessing plant health. The system also captures images of plant leaves, which are processed to assess yellowing levels using color analysis in the HSV (Hue Saturation Value) color space. If yellowing exceeds a threshold of 20%, the data is flagged and sent to a machine learning model. This system offers a low-cost, automated solution for plant care by integrating sensor data, image processing, and machine learning, helping both hobbyists and professionals detect plant health issues early and reduce the need for costly interventions.

Keywords: Machine Learning, Plant Health, Disease Detection, Image Processing, Internet of Things, Monitoring

1. Introduction

Using computers to collect and process plant health data is an ongoing field of study aimed at automating the monitoring and care of plants. The world's population is expected to grow from the current 7.5 billion to 10 billion people by 2050, which indicates a need for sustainable food and agricultural resources (Ali et al., 2024). The current state of research in this field involves utilizing AI (Artificial Intelligence) agents with IoT (Internet of Things) devices, allowing the agent to interact with its environment and act. Current research trends rely on signal and image processing, as well as AI agents, to determine if a plant requires attention. Several automation and smart plant monitoring systems exist; however, these systems do not fully meet customer needs and fail to automatically apply suitable actions upon detecting poor plant conditions (Li et al., 2018).

There have been many contributions to the field over the last 5-10 years, but no feasible solutions currently exist for large-scale applications to monitor individual plants. Recent solutions in the literature for plant health monitoring and care include using automated, AI-powered IoT systems for plant monitoring and actuation (Ali et al., 2024). This solution achieved 75% accuracy for pest detection on chili plants and automatically applied pesticide spray when pests were detected. Hanumann et al. (2022) developed an IoT-based water monitoring system that also records daily temperatures to enable more informed planting strategies for each plot of land. Sathiya, Josephine, and Jeyabalaraja (2022) developed an AI agent to classify herbs and detect diseases using a combination of AI algorithms. Rani, Kumar, and Bhushan (2019) designed an automatic watering system for rice plants, utilizing pH level, temperature, and moisture sensors to detect the need for water in the soil, which is automatically supplied via water pumps (Ali et al., 2024).

Rather than relying solely on image data to detect disease, the proposed system will process both signal and image data to determine if a plant is diseased. The system will include a camera, a soil moisture sensor, a temperature and humidity sensor, and a soil composition sensor to monitor ph. The system will record and store both signal and image data for a plant. The data will be processed at scheduled intervals to determine whether a plant's yellowing leaves are due to disease or another factor, such as underwatering. If the system detects a diseased plant, it will notify the user via a red LED light, indicating the plant is diseased.

The proposed system design and data processing method were selected due to recent contributions in the field that rely either on sensor data alone or AI agents with image processing to automate plant health monitoring. Training an AI agent with both sensor and image data over a historical period could enhance the accuracy of plant disease detection. Automatically and accurately detecting whether a plant's yellowing leaves are caused by disease will help ensure higher yields for plants.

The following sections of this paper will be organized as follows. Section two will describe the related works, and section three will describe the methodology used, along with pseudocode for the algorithms developed and used in this project. Diagrams of the system, along with other images and figures, will also be presented. Section four will conclude the paper with the results, as well as any further research or development that could benefit the project. Finally, the paper will conclude with the references section.

2. Related Works

Ali *et al. (2024) proposed* and built a "an automated AI-powered IoT algorithm with data processing and noise elimination for plant monitoring and actuating" (Ali et al., 2024). The

system utilizes an internet of things (IoT) device and sensors to monitor the plants' climate, as well detect if a plant had pests on it using AI. The system included a temperature and humidity sensor, a soil moisture sensor, CO2 sensor, a camera, pumps for pesticide and water, an enclosure, and a chili plant. Three main different subsystems have been used, namely irrigation, CO2 detecting, and plant infects detection systems (Ali et al., 2024). The irrigation system used a moisture sensor module (Model: SN-MOISTURE-MOD) and if it detects a reading below a certain threshold, it will turn on a pump to add water to the soil (Ali et al., 2024). If the moisture reading is above a certain threshold, then the pump will be shut off stopping the flow of water. The CO2 detection system measures the greenhouses CO2 levels every minute using a CO2 sensor (Model: SN-MQ135-MOD) and a temperature and humidity sensor (Model: DHT22) (Ali et al., 2024). If the reading is above a certain threshold, then a fan is turned on to help circulate the air. If the reading is below an established threshold, then the fan will be turned off. Lastly, the plant infection system utilizes a camera to stream video data of the plant. The video stream is processed frame by frame, using image processing techniques to help determine if black spots on plants' leaves are pests or not, and uses AI to help eliminate noise from sensor readings. If pests are detected, then a pump is turned on that sprays pesticide the kill the pests. The AI models used have a 75% accuracy, but it was stated that more robust sensors would require less noise elimination from the AI models and would be more accurate (Ali et al., 2024).

Gultom *et al. (2017)* developed a smart IoT water sprinkling and monitoring system for a chili plant. The system that was developed used a moisture sensor to measure the moisture levels in the soil of a chili plant. An Arduino was then used to process the data, and based on the reading would open or close a valve to allow that controls the flow of water. The system also uses pH sensor which is used to detect acidity or alkalinity for chili plant and EC sensor to determine the nutrient solution of the soil... the system is also equipped with neutralizing pH and EC automatically (Gultom et al., 2017). This paper focuses on optimizing the use of water as needed of chili plants and Internet of Things (IoT) can be used to data analysis for managed and controlled in detail and precisely by Sensor (Gultom et al., 2017).

Chowdhury *et al. (2021)* used a deep learning model to classify diseases based on tomato leaves. Accuracy scores of 99.95% and 99.12% were achieved using EfficientNet-B7 convolutional neural network (Chowdhury et al., 2021). It was stated that that the performance of their models in the reported work outperforms existing literature.

Hanumann *et al. (2022)* built a smart watering system using IoT devices that would help with increasing productivity and crops by optimizing the use of a water supply. The system would record the daily weather in the local area, current moisture levels of soil, and then process it to determine how much each area needed to be watered, or if it needed to be watered at all based on incoming weather. The collected data in the database is used for machine learning training purposes to help train more accurate models.

Prathibha, Hongal & Jyothi (2017) used IoT devices to monitor temperature and humidity in an agricultural field. A single CC3200 chip, camera, and multiple sensors were used to capture images and send pictures through MMS to farmers using Wi-Fi (Prathibha, Hongal & Jyothi, 2017). The device made was small and portable and used a low voltage power supply. It was ideal for long-term outdoor situations. Users are able to see long accurate changes in crop yield, is cheap to produce, and consumes little power (Prathibha, Hongal & Jyothi, 2017).

In this paper, we focus on using data measured from sensors to train an AI model to determine if a plant is diseased based on the detection of yellow leaves, or if the yellowing leaves are the result of something else such as over or underwatering, or a nitrogen deficiency based on

historical sensor readings of the plants environment. Rather than using a stream of video data and processing it through a machine learning algorithm to detect disease, the proposed system will process an image of the plant at routine intervals to detect if there is yellow on the leaves. Upon detection, the system then will process historical time series data of sensor readings to determine if the yellowing was caused by disease, or some other factors. If the plant needs to be watered, then the watering system will activate a pump to do so. If a nitrogen deficiency is detected, then the system will automatically add fertilizer to the soil, if a disease, then the system will notify the user of a disease detection. This system is aimed at reducing the processing cost of processing 30 frames a second of video through an AI algorithm but instead processing smaller amounts of data and notifying a user if a specific plant needs attention or not if a disease is detected.

3. Methodology

The proposed plant health monitoring system aims to provide a comprehensive solution for assessing the health of plants in real-time by combining environmental sensor data with image processing techniques. The system integrates various hardware components and machine learning models to continuously monitor plant health, diagnose potential diseases, and notify the user when plant care interventions are required.

System Workflow:

1. **Data Collection**: The Raspberry Pi reads data from the sensors (soil moisture, pH, temperature, and humidity) at regular intervals.
2. **Image Capture**: A photo of the plant is captured and passed to the YellowDetector module for processing.
3. **Yellowing Detection**: The YellowDetector module calculates the percentage of yellowing on the leaves.
4. **Data Flagging**: If the yellowing exceeds the 20% threshold, the data entry is flagged for further analysis.
5. **Health Diagnosis**: The HealthModel processes the sensor data and yellowing percentage to predict if the yellowing is due to environmental factors or a disease.
6. **User Notification**: If a disease is detected, the red LED is activated for 10 seconds to notify the user.

3.1 System Hardware Configuration

The system is built around a Raspberry Pi4 that serves as the central computing unit. The Raspberry Pi interfaces with several sensors that measure critical environmental factors affecting plant health. The sensors are connected to the Raspberry Pi via GPIO pins, which reads the values at specified intervals. This data is then stored and processed for further analysis. These sensors include:

- **LM393 Soil Moisture Detection Sensor**: Measures the moisture content in the soil to determine if the plant is adequately hydrated. Soil moisture is a key factor in plant growth and can be an early indicator of dehydration or over-watering.
- **Teyleten Robot pH Value Data Detection Sensor**: Measures the pH level of the soil, which is important for understanding whether the soil is acidic or alkaline. An imbalance in pH can impede nutrient absorption and affect plant health.

- **DHT22/AM2302 Digital Humidity and Temperature Sensor**: Captures ambient temperature and humidity levels around the plant. These environmental parameters play a crucial role in determining the overall health and stress levels of plants.
- **MSP3008 Analog-to-Digital Converter (ADC)**: Used to convert the analog output from the pH sensor into a digital signal that the Raspberry Pi can process.

```
Initialize sensors, camera, data handler, yellowing detector,
and health model
Set monitoring interval and LED pin
Configure GPIO and turn LED off

Loop forever:
    Read temperature, humidity, moisture, and pH values
    Capture image using camera
    Analyze image for yellowing percentage

    If yellowing ≥ 20%:
        Mark entry as yellowing = 1
    Else:
        yellowing = 0

    Create data entry with sensor readings, image name, and
yellowing status
    Save data entry

    If yellowing == 1:
        Predict disease using health model
        If disease detected:
            Turn on LED for 10 seconds

    Release the camera
    Wait for the specified interval
    Reinitialize the camera
```

Table 1. Main Module Algorithm Pseudocode

3.2 Image Capture and Processing

In addition to environmental monitoring, the system uses a USB camera to capture images of the plant periodically. The captured images are processed by an image processing module called YellowDetector. The YellowDetector module is designed to process images of plant leaves and quantify the percentage of yellowing, a key indicator of plant health. The image processing steps involved in detecting yellowing are as follows:

1. **HSV Conversion**: The image is first loaded and converted from the standard BGR color space to the HSV (Hue, Saturation, Value) color space using OpenCV's cv2.cvtColor. This transformation helps to more easily isolate color ranges for yellow and green tones, which are critical for detecting yellowing.
2. **Mask Creation:**
    a. Green Mask: A mask is created for the green pixels in the image using the defined lower and upper green color boundaries. This helps to segment the plant's healthy, non-yellow parts.
    b. Yellow Mask: Similarly, a mask is created to isolate the yellow pixels using the defined yellow color boundaries.
3. **Combining Masks:** The green and yellow masks are combined using a bitwise OR operation, allowing the system to focus on both the green and yellow regions of the image. This combined mask ensures that the analysis focuses only on the plant's relevant areas (the leaves), ignoring the background or other unrelated parts of the image.
4. **Yellow Pixel Counting**: The number of yellow pixels is calculated by summing the pixels in the yellow mask where the value is 255, indicating the presence of yellow.
5. **Total Pixel Counting**: The total number of pixels in the combined mask (yellow and green) is calculated by summing the pixels where the combined mask value is 255.
6. **Yellow Percentage Calculation**: The percentage of yellow pixels is calculated by dividing the number of yellow pixels by the total number of pixels in the combined mask, then multiplying by 100 to get the percentage of yellowing in the image.
7. **Edge Case Handling:** If no relevant pixels are detected (i.e., the total pixels in the combined mask are zero), the function returns a yellowing percentage of 0.0 to avoid division by zero errors.

The module then returns the percentage of yellowing detected in the plant's leaves. This percentage is used to assess the health of the plant, with yellowing above a certain threshold indicating potential stress or disease.

```
Initialize YellowDetector:
     Define HSV color range for yellow and green

Function detect_yellowing(image_path):
     Load image from given path
     If image fails to load:
          Raise error

     Convert image to HSV color space

     Create mask for green areas using green HSV range
     Create mask for yellow areas using yellow HSV range
     Combine green and yellow masks

     Count yellow pixels (where yellow_mask == 255)
     Count total pixels in combined mask

     If total_pixels == 0:
```

```
          Return 0.0 (to avoid division by zero)

     Calculate yellow_percentage = (yellow_pixels /
total_pixels) * 100

     Return yellow_percentage
```

Table 2. YellowDetector Pseudocode Algorithm

3.3 Training of the HealthModel

The HealthModel is trained on a synthetic dataset consisting of 10,000 data points, each representing a combination of sensor readings (soil moisture, pH, temperature, and humidity) and the corresponding level of yellowing detected in the plant leaves. These data points are generated to simulate various environmental conditions and their effects on plant health, particularly focusing on the miniature fig tree. The XGBClassifier was chosen for its efficiency in handling large datasets and its ability to produce accurate classification results. The model was trained to classify whether yellowing was caused by environmental factors or by other potential causes, such as disease.

The generated dataset is split into training and testing sets where 80% of the data is used for training the model and 20% for evaluating its performance. The train_model method utilizes the XGBClassifier (XGBoost algorithm) to train a model on the training dataset. The classifier is fine-tuned with parameters such as colsample_bytree, learning_rate, and max_depth to ensure robust learning and to handle class imbalance (e.g., more instances of no yellowing than yellowing). After training, the model is saved to a file for future use.

```
Function generate_data():
     Initialize empty data list
     Set current_date to starting date

     For each record_id in number of records:
          Calculate days_from_start = record_id
          Generate temperature, humidity, moisture, and pH
using seasonal trends
          Simulate yellowing based on sensor readings

          Create data entry with:
               - timestamp = current_date
               - temperature, humidity, moisture, pH
               - yellowing status

          Append data entry to data list
          Increment current_date by one day

     Return data as a structured DataFrame
```

Table 3. Training Data Generation Algorithm Pseudocode

```
Function train_model(X_train, y_train):
     Initialize XGBoost classifier with:
          - colsample_bytree = 0.8
          - learning_rate = 0.01
          - max_depth = 5
          - min_child_weight = 1
          - n_estimators = 200
          - subsample = 0.8
          - scale_pos_weight = 3 (to handle class imbalance)

     Train the model using X_train and y_train

     Save the trained model to file

     Assign trained model to instance variable
```

Table 4. HealthModel Training Algorithm Pseudocode

```
Function evaluate_model(X_test, y_test):
     Predict labels (y_pred) using the trained model on X_test
     Calculate accuracy between y_test and y_pred
     Display test accuracy

     Generate and display:
          - Classification report
          - Confusion matrix
```

Table 5. HealthModel Evaluation Algorithm Pseudocode

```
Function test_model(optimal_points = 100, non_optimal_points =
100):
     Generate optimal condition data
     Generate non-optimal condition data
     Combine both datasets into full_data

     Extract feature columns (excluding 'timestamp' and
'yellowing') as X_pred
     Extract actual yellowing status as y_pred_actual

     Predict labels using trained model on X_pred

     For predictions in optimal_data:
          Count correct predictions
```

```
     For predictions in non_optimal_data:
          Count correct predictions

     Display number of correct predictions for:
          - Optimal conditions
          - Non-optimal conditions
```

Table 6. HealthModel Testing Algorithm Pseudocode

3.4 Data Analysis and Health Diagnosis

Once the yellowing percentage is calculated, the data is passed to the HealthModel, a machine learning model trained to diagnose plant health based on both environmental sensor data and leaf yellowing. The model uses an XGBClassifier, which is well-suited for classification tasks. The HealthModel is trained on a synthetic dataset containing 10,000 data points, each consisting of environmental sensor readings (soil moisture, pH, temperature, and humidity) and corresponding yellowing levels of the plant.

The model predicts whether the yellowing is caused by environmental factors or not, considering both the sensor readings and the yellowing percentage. If the system detects significant yellowing and finds that the environmental conditions (e.g., soil moisture, pH, or temperature) are outside the optimal range for the miniature fig tree (the plant of choice in this study), the model diagnoses the plant as potentially diseased.

```
Function predict_disease(entry):
     Convert entry dictionary into a single-row DataFrame
     Remove 'timestamp', 'yellowing', and 'image_name' columns
to prepare features (X_pred)

     Predict yellowing status using the trained model

     If model predicts no yellowing AND actual yellowing is 1:
          Extract temperature, humidity, moisture, and pH from
entry
          If environmental conditions are optimal:
               Return 1 (disease likely present)

     Return 0 (no disease detected)
```

Table 7. HealthModel Prediction Algorithm Pseudocode

3.5 User Notification and Feedback

If a disease is detected, the system activates a red LED light on the breadboard for 10 seconds to alert the user that the plant may be under stress or diseased. This feedback mechanism provides the user with a quick and intuitive signal to investigate the plant's condition and take appropriate action, such as adjusting watering, soil pH, or environmental factors.

4. Implementation and Results

The system was developed using a combination of hardware and software components to ensure robust monitoring and analysis. The core hardware consists of a **Raspberry Pi 4**, which serves as the central computing unit for data collection, processing, and analysis. The Raspberry Pi is interfaced with multiple sensors that provide environmental data.

4.1 Hardware Components:

- **LM393 Soil Moisture Sensor**: Measures soil moisture levels to assess irrigation needs.
- **Teyleten Robot pH Sensor**: Measures soil pH, which is critical for plant health.
- **DHT22/AM2302 Digital Humidity and Temperature Sensor**: Provides real-time temperature and humidity readings.
- **MSP3008 Analog-to-Digital Converter**: Converts the analog signal from the pH sensor into a digital format suitable for processing by the Raspberry Pi.

A camera module is also attached to the Raspberry Pi to capture images of plant leaves at regular intervals for image processing.

4.2 Software Components and Libraries:

- **Python**: The primary programming language used to develop the system due to its extensive support for hardware interaction and machine learning.
- **OpenCV**: Used for image processing tasks, specifically the YellowDetector module, which analyzes the leaf images to detect yellowing using HSV color space conversion and pixel masking techniques.
- **Scikit-learn** and **XGBoost**: Used for developing and training the machine learning model. The **XGBoost classifier** was selected due to its robustness and effectiveness in handling imbalanced datasets.
- **Pandas** and **NumPy**: Used for data manipulation, including creating synthetic datasets and processing sensor data.
- **Raspbian OS**: The operating system running on the Raspberry Pi, which supports all the necessary Python libraries and tools.

4.3 Data Type and Sources

The system collects environmental data from the sensors, which includes the following types:

- **Temperature** (°C)
- **Humidity** (%)
- **Soil Moisture** (percentage of soil moisture)
- **pH** (value of soil pH)

4.4 Methods Used

The YellowDetector module analyzes captured leaf images to detect yellowing. The images are converted to the HSV color space to distinguish yellow and green pixels, as yellowing typically manifests as a shift in the leaf's color. The following steps are performed:

1. **HSV Conversion**: Convert the captured RGB image to HSV.
2. **Masking**: Create masks for yellow and green pixel ranges.
3. **Pixel Counting**: Calculate the percentage of yellow pixels in relation to the total number of pixels in the image.
4. **Thresholding**: If the percentage of yellow pixels exceeds a predefined threshold (20%), the data entry is flagged for yellowing.

4.4 Experiment Setup

For testing and validation purposes, a controlled environment was set up to simulate various plant conditions. Data was collected over several days with different sensor readings, and images were captured at intervals to assess the effectiveness of the YellowDetector module in detecting yellowing. The synthetic data was used for training the model, while real-world data was used for testing the system's performance.

Additionally, experiments with optimal and non-optimal conditions were performed to test the system's ability to distinguish between environmental stress and disease. The model was tested with specific data points representing both conditions, and accuracy was measured by comparing predicted and actual yellowing outcomes. Figure 9 shows the physical hardware of the system connected and running.

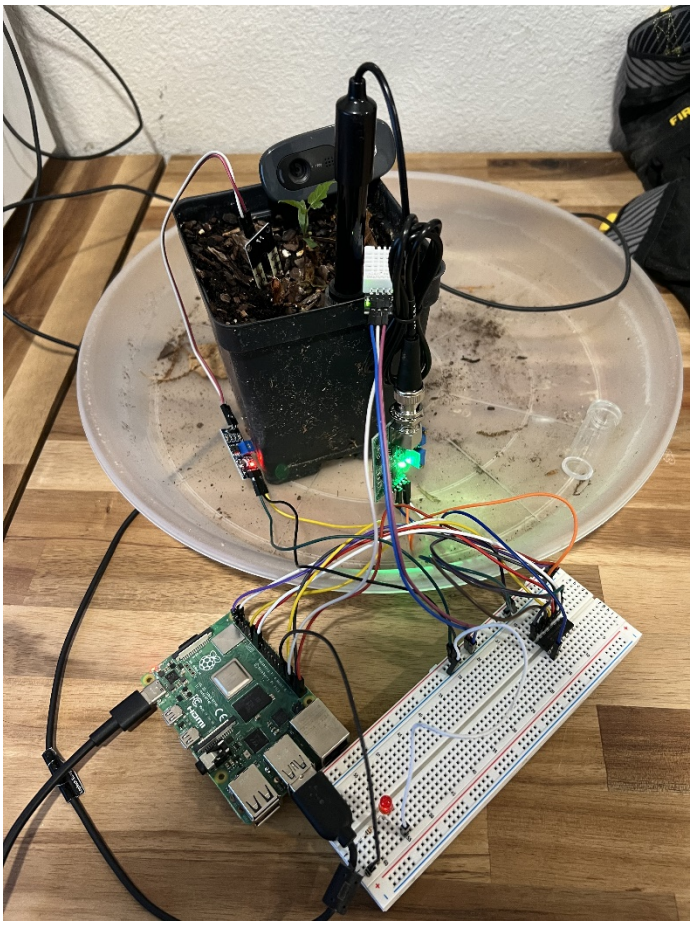


Figure 1. Health Monitor Physical System

5. Results

The performance of the health model, trained on synthetic data generated from various environmental conditions, was evaluated using a test dataset consisting of 200 samples. The model achieved a test accuracy of 86%, indicating a strong ability to correctly classify plant health conditions based on sensor data and yellowing levels.

5.1 Model Performance

The classification report, presented in Table 1, shows that the model performs well across both classes—healthy plants (class 0) and plants with yellowing (class 1).

```
Test Accuracy: 0.8600
Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.83      0.89       142
           1       0.69      0.93      0.79        58

    accuracy                           0.86       200
   macro avg       0.83      0.88      0.84       200
weighted avg       0.89      0.86      0.86       200
```

Figure 2. Classification Report

For class 0 (healthy plants): The model achieved a high precision of 0.97 and a recall of 0.83, leading to an F1-score of 0.89. This indicates that the model is highly accurate in predicting healthy plants, though it misses some yellowing plants (indicated by the recall value of 0.83).

For class 1 (yellowing plants): The model achieved a precision of 0.69 and a recall of 0.93, resulting in an F1-score of 0.79. Although the precision for yellowing detection is lower, the high recall suggests that the model is effective in identifying most plants exhibiting yellowing, but with a higher risk of false positives.

5.2 Confusion Matrix Analysis

The confusion matrix, shown in Figure 3, provides further insight into the model's performance:

```
Confusion Matrix:
[[118  24]
 [  4  54]]
```

Figure 3. Confusion Matrix

- **True positives (54)**: The model correctly identified 54 instances of yellowing plants.
- **True negatives (118)**: The model correctly identified 118 instances of healthy plants.
- **False positives (24)**: The model incorrectly classified 24 healthy plants as yellowing.
- **False negatives (4)**: The model incorrectly classified 4 yellowing plants as healthy.

The relatively low number of false negatives (4) demonstrates the model's ability to identify yellowing plants effectively, while the false positives (24) suggest that some healthy plants may be wrongly flagged as exhibiting yellowing due to environmental conditions or sensor inaccuracies.

5.3 YellowDetector Performance

The results from the YellowDetector depend on the input image and how much yellowing is present in the leaves. It outputs the percentage of yellowing detected. The detection process is visually represented by displaying the original image (Figure 4), yellow mask (Figure 5), green

mask (Figure 6), and combined mask (Figure 7), allowing users to see how the algorithm isolates the yellow regions. If a significant amount of yellowing is detected (more than 20% yellowing), then the sensor and image data is sent to the HealthModel for further processing to determine if yellowing is due to disease.



Figure 4. Original Image

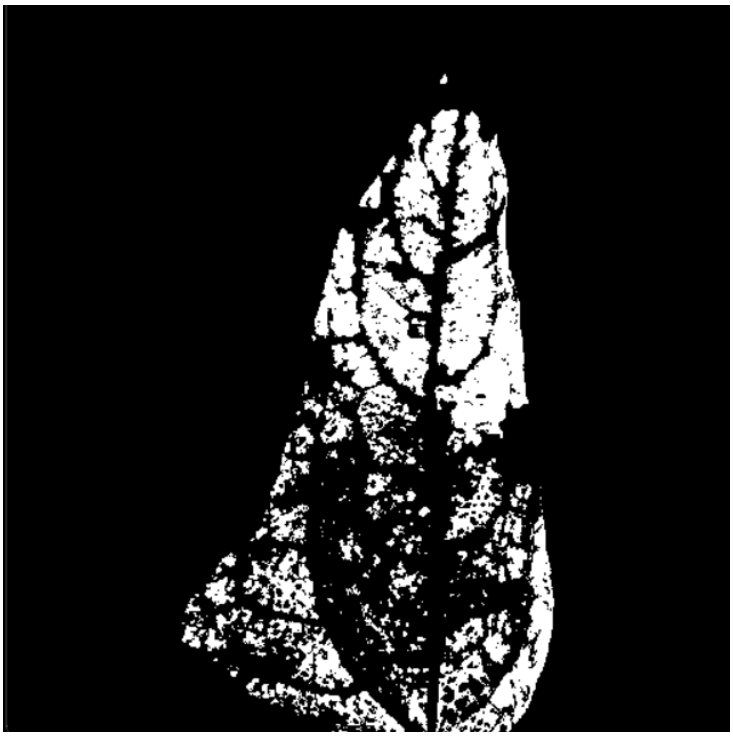


Figure 5. Yellow Mask

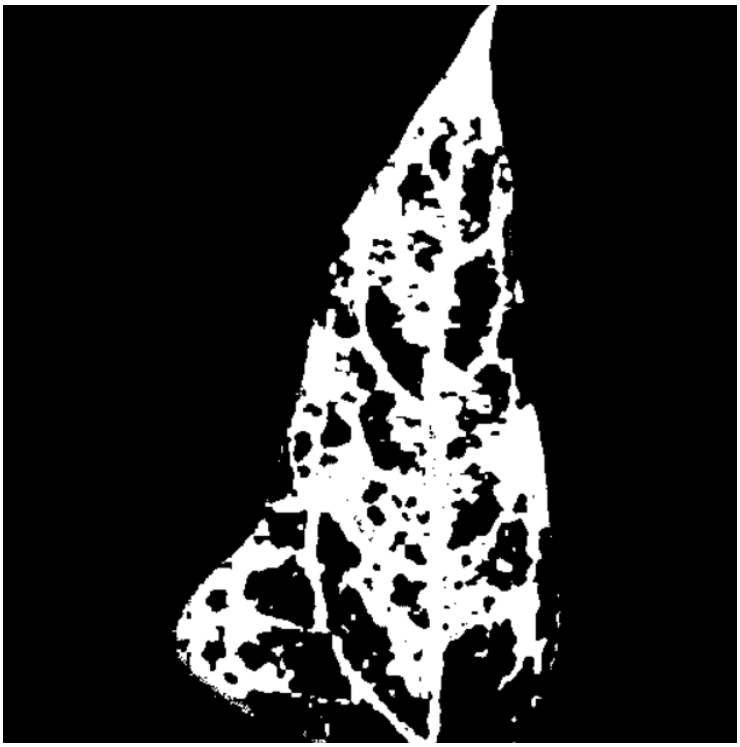


Figure 6. Green Mask



Figure 7. Combined Mask

6. Future Work and Discussion/Conclusion

The plant health monitoring system presented in this study demonstrates a solid foundation for tracking the well-being of plants using a combination of environmental sensors and image analysis techniques. The integration of the LM393 soil moisture sensor, Teyleten Robot pH value sensor, DHT22/AM2302 humidity and temperature sensor, and a Raspberry Pi-based control unit successfully collects and processes the necessary data to assess plant health. However, as highlighted by the system's current 83% accuracy, there is considerable room for improvement in both the HealthModel's predictive capabilities and the sensor reliability. The model's accuracy can be enhanced through additional data, hyperparameter tuning, and more sophisticated machine learning techniques. Likewise, addressing the issues with sensor accuracy, especially in terms of pH readings, will significantly improve the system's robustness and dependability. The addition of an image detection model will further enhance the system's diagnostic capabilities by allowing for direct leaf health analysis. This approach could be expanded to cover a wider range of plants, making the system more versatile. Furthermore, developing a more comprehensive notification system will ensure that users are promptly alerted in case of disease detection, leading to timely interventions and better plant care.

In conclusion, the plant health monitoring system represents a promising step toward smarter, more efficient plant care, combining sensor data with advanced machine learning to provide actionable insights. With continued refinement and expansion, the system has the potential to become a valuable tool for both hobbyist gardeners and professional agriculturalists, promoting healthier plants and more sustainable growing practices.

References

Ali, M. A. H., Moiduddin, K., Nukman, Y., Abd Razak, B., Aboudaif, M. K., & Thangaraj, M. (2024). An automated AI-powered IoT algorithm with data processing and noise elimination for plant monitoring and actuating. *PeerJ. Computer Science*, *10*, e2448-. https://doi.org/10.7717/peerj-cs.2448

A. Kumar, K. Kamal, T. Vadamala and S. Mathavan 2014. "Smart Irrigation Using Low-cost Moisture Sensors And Xbee-based Communication", *IEEE Global Humanitarian Technology Conference*, 2014.

A. Bah, S.K. Balasundram and M.H.A. Husni 2014. "Sensor Technologies for Precision Soil Nutrient Management and Monitoring", *American Journal of Agricultural and Biological Sciences*, vol. 7, no. 1, pp. 43-49, 2012.

Arya, S.; Singh, R. 2019. A Comparative Study of CNN and AlexNet for Detection of Disease in Potato and Mango leaf. In Proceedings of the 2019 International Conference on Issues and Challenges in Intelligent Computing Techniques (ICICT), Ghaziabad, India, 27–28 September 2019; pp. 1–6.

AshifuddinMondal M, Rehena Z. 2018. IoT based intelligent agriculture field monitoring system. In: 2018 8th International Conference on Cloud Computing, Data Science & Engineering (Confluence). Noida, India, 625–629.

Ashwini Raut, Mrunal Panse, Darshana Chaware and Aniruddha Koparkar 2015. "Sensor Based Automated Irrigation System", *International Journal of Engineering Research & Technology*, vol. 4, no. 05, May 2015, ISSN 2278-0181.

Chowdhury MEH, Rahman T, Khandakar A, Ayari MA, Khan AU, Khan MS, Al-Emadi N, Reaz MBI, Islam MT, Ali SHM. 2021. Automatic and reliable leaf disease detection using deep learning techniques. AgriEngineering 3(2):294–312 DOI 10.3390/agriengineering3020020.

Gultom JH, Harsono M, Khameswara TD, Santoso H. 2017. Smart IoT water sprinkle and monitoring system for chili plant. In: 2017 International Conference on Electrical Engineering and Computer Science (ICECOS). Palembang, Indonesia, 212–216 DOI 10.1109/ICECOS.2017.8167136.

Hanumann T, Swamy SN, Gowtham P, Sumathi R, Chinnasamy P, Kalaiarasi A. 2022. Plant monitoring system cum smart irrigation using Bolt IoT. In: 2022 International Conference on Computer Communication and Informatics (ICCCI). Coimbatore, India, 1–3.

Hui Liu, Zhijun Meng and Shuanghu Cui 2007. "A wireless sensor network prototype for Environmental monitoring in greenhouses", *Proceedings of the International Conference on Wireless Communications Networking and Mobile Computing 2007 (WiCom 2007) 21–25 Sept. 2007*, pp. 2344-2347, 2007.

I. Mampentzidou, E. Karapistoli and A.A. Economide 2012. "Basic Guidelines for Deploying Wireless Sensor Networks in Agriculture", *Fourth International Workshop on Mobile Computing and Networking Technologies*, pp. 864-869, 2012.

Kamilaris, A., F. Gao, F.X. Prenafeta-Boldu, and M.I. Ali (2016). Agri-IoT: A semantic framework for Internet of Things-enabled smart farming applications. in 2016 IEEE 3rd World Forum on Internet of Things (WF-IoT). , pp 442–447

Kshitij Shinghal, Arti Noor, Neelam Srivastava and Raghuvir Singh 2011. "intelligent humidity sensor for wireless sensor network agricultural application", *International Journal of Wireless & Mobile Networks (IJWMN)*, vol. 3, no. 1, February 2011.

Li, Y., Ding, Y., Li, D., & Miao, Z. 2018. Automatic carbon dioxide enrichment strategies in the greenhouse: A review. Biosystems Engineering, 171, 101–119. https://doi.org/10.1016/j.biosystemseng.2018.04.018

Mohamed Rawidean Mohd Kassim, Ibrahim Mat and Ahmad Nizar Harun 2014. "Wireless Sensor Network in Precision agriculture application" in International conference on computer Information and telecommunication systems (CITS), published in IEEE Xplore, July 2014.

Mohamood N, Zainal N, Kadmin AF, Gani SFA. 2019. Integrated smart farming based on internet of things (IoT) system for figs cultivation. International Journal of Innovative Technology and Exploring Engineering 9(2):1429–1436 DOI 10.35940/ijitee.B6157.129219.

Mohanraj I, Kirthika Ashokumar JN. 2016. Field monitoring and automation using IoT in agriculture domain. Procedia Computer Science 93(8):931–939

Nikesh Gondchawar and R. S. Kawitkar 2016. IoT based Smart Agriculture, *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 5, no. 6, pp. 2278-1021, June 2016.

Prathibha SR, Hongal A, Jyothi MP. 2017. IoT based monitoring system in smart agriculture. In: 2017 International Conference on Recent Advances in Electronics and Communication Technology (ICRAECT). Bangalore, India, 81–84.

Rani D, Kumar N, Bhushan B. 2019. Implementation of an automated irrigation system for agriculture monitoring using IoT communication. In: 2019 5th International Conference on Signal Processing, Computing and Control (ISPCC). Solan, India, 138–143.

Sathiya, V., Josephine, M. S., & Jeyabalaraja, V. 2022. An automatic classification and early disease detection technique for herbs plant. *Computers and Electrical Engineering*, *100*, 108026. https://doi.org/10.1016/j.compeleceng.2022.108026

Saraswathi D, Manibharathy P, Gokulnath R, Sureshkumar E, Karthikeyan K. 2018. Automation of hydroponics green house farming using IoT. In: 2018 IEEE International Conference on System, Computation, Automation and Networking (ICSCA). Pondicherry, India

Stefanos A. Nikolidakis, Dionisis Kandris, D. Dimitrios and Douligeris A Vergadoschristos. 2015. Energy Efficient Automated Control Of Irrigation In Agriculture By Using Wireless Sensor Networks Computers And Electronics In Agriculture" in ????, Elsevier B.V, pp. 0168-1699, 2015.

S. Li, J. Cui and Z. Li 2011. Wireless Sensor Network for Precise Agriculture Monitoring, *Fourth International Conference on Intelligent Computation Technology and Automation*, March 28–29, 2011.

Tan, M.; Le, Q 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In Proceedings of the International Conference on Machine Learning, Long Beach, CA, USA, 10–15 June 2019; pp. 6105–6114.

Tanmay Baranwal and Nitika Pushpendra Kumar Pateriya, 2016. Development of IoT based Smart Security and Monitoring Devices for Agriculture, in 6th International Conference - Cloud System and Big Data Engineering, IEEE, pp. 978–1-4673-8203-8/16, 2016.

Vaidya S, Ambad P, Bhosle S. 2018. Industry 4.0–a glimpse. Procedia Manufacturing 20(1):233– 238 DOI 10.1016/j.promfg.2018.02.034.